



Text-to-SQL Dashboard

Complete Documentation

Ask your database questions in plain English and get back validated, read-only SQL, a results table, and an auto-picked chart — powered by a fully local LLM stack.

 Ollama (local LLM)

 LangGraph

 ChromaDB

 SQLAlchemy

 sqlglot validation

 Streamlit + Plotly

Covers how the application works, how to set it up and configure it, and how to use it day to day.

For a deep, code-level technical walkthrough, see [docs/ARCHITECTURE.md](#).

CONTENTS

Table of Contents

- [1 Overview & Key Features](#)
- [2 How It Works](#)
- [3 Project Structure](#)
- [4 Getting Started \(Install & Configure\)](#)
- [5 Using the Dashboard](#)
- [6 Under the Hood: Retrieval, Retries & Safety](#)
- [7 Security & Data Privacy](#)
- [8 Testing & Quality Checks](#)
- [9 Known Limitations](#)
- [10 Troubleshooting](#)
- [11 Frequently Asked Questions](#)
- [12 Glossary](#)
- [13 Further Reading](#)

Overview & Key Features

What this app does

This dashboard lets you ask questions about a real, live database in plain English — no SQL required. You type a question, the app writes a SQL query for you, shows it to you for review, and only ever runs it once you approve. Results appear as a table, with a chart drawn automatically when the data shape supports one.

Every query is read-only. The app can look at your data but is never allowed to change it — this is enforced in code, not just by convention (see Chapter 7).

A typical flow



Who it's for

Anyone who needs answers from a database but doesn't write SQL day to day — analysts, product managers, support staff — as well as engineers who want a fast, inspectable way to explore a schema they don't already have memorized.

Key features

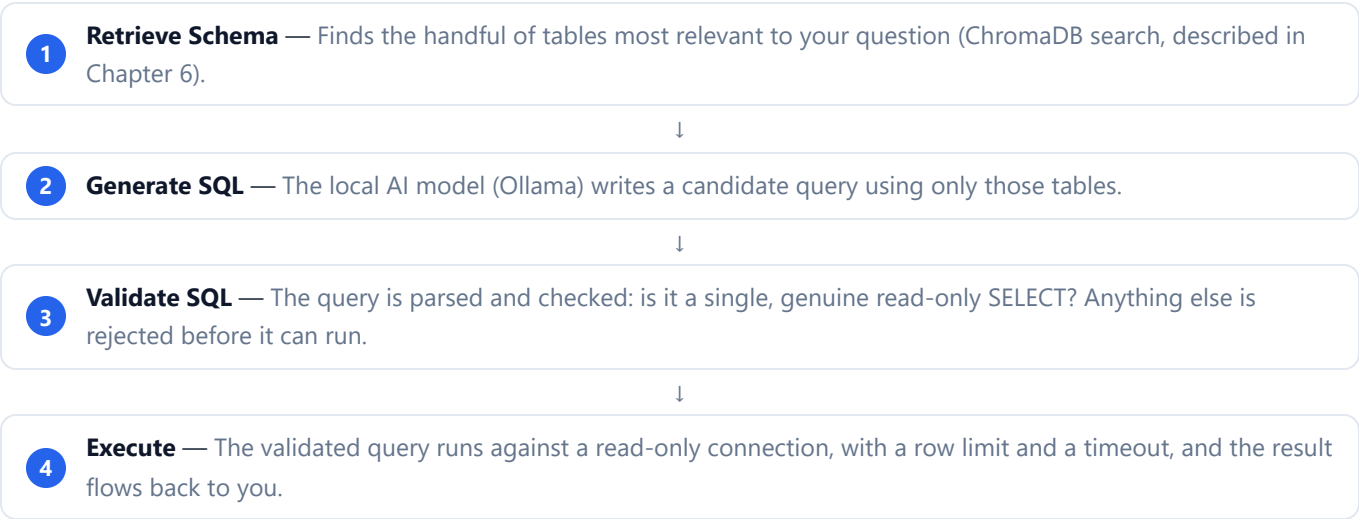
Feature	What it means for you
Self-correcting answers	If the first SQL attempt has a mistake, the app automatically tries again using the specific error — up to a capped number of attempts — instead of just failing.
Read-only by construction	Every query is checked against an allowlist that accepts only <code>SELECT</code> -style statements. Nothing your questions do can insert, update, delete, or alter data.
Schema-aware, not schema-dumping	The AI model never sees your entire database schema at once. It's shown only the handful of tables relevant to your specific question — this keeps answers accurate even on databases with hundreds of tables.

Works with your real database	PostgreSQL, MySQL, SQL Server, or Oracle — configured entirely through a settings file, with no sample or hardcoded data anywhere.
Human-readable results	Internal ID/key columns are hidden from the default view, and column names are expanded into plain language (e.g. <code>CustName</code> → "Customer Name") — without ever changing the underlying query.
Curated business context	A hand-maintained notes file describes what each table means in plain language (e.g. "this code column isn't a category, it's an internal flag") — the AI reads it fresh on every question, so a correction takes effect immediately.
Fully local AI	The language model runs on your own machine via Ollama. No API keys, no per-question cost, and no data — schema or results — ever leaves the machine.
Transparent retries	Every attempt the app makes, successful or not, is recorded and viewable in a "Retry timeline" — nothing about the self-correction process is hidden.
Built-in accuracy checks	A growable library of real test questions with pass/fail checks (row counts, readable output, correct tables joined) catches accuracy regressions before they ship.

Why it works this way: most "chat with your database" tools either trust the AI's SQL blindly, or hide its reasoning behind an opaque black box. This app treats AI output as untrusted by construction — every query is parsed and checked before it can run, every retry is visible, and the schema shown to the model scales to a real production database instead of assuming a small toy dataset.

How It Works

Under the hood, the app is a small, explicit four-step pipeline (a "state machine") rather than a free-form AI agent that decides its own next move. That's a deliberate design choice: every possible path — including every retry — is a fixed, readable step in a diagram, not something that emerges unpredictably from the model's own reasoning.



If step 3 or step 4 fails in a way that's worth retrying, the app loops back to step 1 or step 2 with the specific error in hand — up to a configurable limit — rather than giving up on the first mistake. Chapter 6 explains exactly which failures get retried and which don't.

The technology underneath

Layer	Choice	Why this one
AI / LLM runtime	Ollama (default model <code>llama3.1:8b</code>)	Runs entirely on your machine — no API keys, no data leaving the network, no per-question cost.
Orchestration	LangGraph	An explicit, inspectable state machine (Chapter 2's diagram) instead of a free-form agent loop.
Schema retrieval	ChromaDB	A local vector search index that finds the small set of relevant tables instead of dumping the whole schema into every question.
Database access	SQLAlchemy	One consistent connection layer across four supported databases, driven entirely by configuration.
SQL safety check	sqlglot	Parses the actual query structure and allowlists what's permitted, rather than scanning for "bad words" in the text (which is easy to trick).

Interface	Streamlit + Plotly	A fast, reviewable web UI — editable SQL box, retry timeline, schema browser — without a separate frontend build.
-----------	---------------------------	---

In one sentence: your question never talks to the database directly — it goes through a model that only sees a relevant slice of the schema, produces SQL that is independently verified before it can run, and every step of that journey is visible to you in the UI.

Project Structure

You don't need to read code to use the app, but this map is useful if you're curious where a particular behavior lives, or if you're evaluating the project technically.

Folder	What lives here
agent/	The AI pipeline: the four-step graph (<code>graph.py</code> , <code>nodes.py</code>), the SQL safety allowlist (<code>sql_validator.py</code>), and error classification for the retry logic.
config/	All tunable settings in one place (<code>settings.py</code>) — model name, database connection fields, row limits, timeouts — plus the hand-maintained table-meaning notes (<code>table_descriptions.yaml</code>) and column-label rules.
db/	Database connection lifecycle and live schema introspection — the app reads your actual tables/columns/relationships at runtime, never from a hardcoded copy. Also holds the "sample real values from coded columns" logic.
embeddings/	Builds and searches the ChromaDB index that powers schema-aware retrieval (Chapter 6).
ui/	The Streamlit web app (<code>app.py</code>) — the only place that renders anything on screen — plus the result-formatting helpers that make columns human-readable.
scripts/	Standalone command-line tools: verify your database connection, build the schema index, and run the accuracy-eval harness.
tests/	The automated test suite — fully mocked, so it runs without a real database or Ollama.
eval/	Real natural-language test questions with pass/fail expectations, used to catch accuracy regressions (Chapter 8).
docs/	This document and the deeper technical architecture write-up.

One rule worth knowing: nothing in this project hardcodes a database connection, host, or schema. Everything about *your* database — including which tables exist — is discovered live at startup and refreshed on demand. See Chapter 4 for how to point it at your own database.

Getting Started

Prerequisites

- **Python 3.11** is the project's target version (see the note below if your machine only has a newer interpreter).
- **Ollama**, installed and running, with a model pulled — `ollama pull llama3.1:8b`.
- **A database to connect to** — PostgreSQL, MySQL, SQL Server, or Oracle. No sample database is bundled with the project by design (see the callout below for a way to try it without one).

Install (Windows / PowerShell)

```
git clone <this-repo-url>
cd Gen_AI_Project_TSQL
py -3.11 -m venv .venv          # or: python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Copy-Item .env.example .env
```

macOS/Linux/WSL users: the equivalent bash commands are in the README — the only difference is venv activation (`source .venv/bin/activate`) and using `cp` instead of `Copy-Item`.

Python version note: if your machine only has a newer Python (e.g. 3.14) and no 3.11, the project still runs — the virtual environment simply gets created against whatever interpreter is available. If a pinned dependency ever lacks a prebuilt wheel for your interpreter version, that mismatch is why.

Configure your database connection

Edit the new `.env` file with your real connection details. Every field is documented inline in `.env.example`; the ones you'll always set are:

Setting	Default	Meaning
<code>DB_TYPE</code>	—	One of <code>postgresql</code> , <code>mysql</code> , <code>mssql</code> , <code>oracle</code> .
<code>DB_HOST</code>	—	Server hostname or IP.
<code>DB_PORT</code>	engine default	5432 / 3306 / 1433 / 1521 respectively if left blank.
<code>DB_NAME</code>	—	Database (catalog) name.
<code>DB_USER</code> / <code>DB_PASSWORD</code>	—	Login credentials — use a dedicated read-only account (see Chapter 7).
<code>DB_SCHEMA</code>	connection default	Optional — restrict what the app can see to one schema.

<code>DB_CONNECTION_STRING</code>	unset	Optional full override; if set, all the discrete <code>DB_*</code> fields above are ignored.
<code>DB_ODBC_DRIVER</code>	ODBC Driver 17 for SQL Server	SQL Server only — must match a driver actually installed on this machine.

Less commonly changed settings:

Setting	Default	Meaning
<code>OLLAMA_HOST</code>	<code>http://localhost:11434</code>	Where Ollama is running.
<code>OLLAMA_MODEL</code>	<code>llama3.1:8b</code>	Swap for a SQL-specialized model (e.g. <code>sqlcoder</code> , <code>duckdb-nsql</code>) or a larger/smaller one.
<code>SCHEMA_TOP_K</code>	<code>4</code>	How many tables are retrieved per question before FK-bridge expansion.
<code>MAX_RETRIES</code>	<code>3</code>	Maximum self-correction attempts before the app gives up on a question.
<code>MAX_RESULT_ROWS</code>	<code>1000</code>	Hard cap on rows returned by any query.
<code>QUERY_TIMEOUT_SECONDS</code>	<code>15</code>	Wall-clock limit before a running query is stopped.
<code>LOG_LEVEL</code>	<code>INFO</code>	Terminal log verbosity.

Trying it without a database of your own: this project was built and tested against Microsoft's public **AdventureWorksDW2025** sample data warehouse (SQL Server). It can be downloaded and restored from Microsoft's official samples page — any edition of SQL Server, including the free LocalDB/Express editions, works.

Verify, build, and run

```
# 1. Confirm .env is correct before doing anything else
python scripts\test_db_connection.py

# 2. Introspect the schema and build the search index (repeat any time
#    the schema changes -- also available from the app's "Refresh Schema" button)
python scripts\build_embeddings.py

# 3. Launch the app
streamlit run ui\app.py
```

PowerShell shortcuts for all of the above are in `tasks.ps1` (`.\tasks.ps1 setup`, `.\tasks.ps1 run`, `.\tasks.ps1 check-db`, `.\tasks.ps1 embed`). Equivalent `make` targets exist for macOS/Linux/WSL.

The app opens in your browser, normally at `http://localhost:8501`. If the database can't be reached, you'll see a clear "Database Connection Required" screen instead of a broken app — fix `.env` and restart.

Using the Dashboard

The top bar

The header shows which AI model is active, which database/type you're connected to, and how many tables were discovered — a quick at-a-glance sanity check that you're pointed at the right place.

The sidebar: database connection

- **Type & Database** — which database engine and database name you're connected to.
- **User** — your database login, partly masked for privacy (e.g. `s***a`).
- **Test Connection** — click any time to re-check the connection live and get a clear pass/fail message.
- **Refresh Schema** — re-scans the database's tables and rebuilds what the app knows. Use this after tables or columns change; it's quick if nothing actually changed.
- **Discovered tables** — an expandable list of every table and column the app can see and use to answer questions. Useful for checking whether the data you're asking about is actually available.

Asking a question

Type your question into the chat box in plain English and press Enter. A status message describes what's happening — retrieving schema, generating SQL, self-correcting if needed.

Tips for better questions

- Be specific: "total sales by year" works better than "show me sales."
- Name entities the way they appear in your data where you can (a category, region, or product name).
- If a question is broad or ambiguous, the app makes a reasonable assumption — check the generated SQL to see what it decided, and re-ask more specifically if that wasn't what you meant.
- If you ask about something that isn't in the database at all, the app says so rather than making up an answer.
- Asking the exact same question again within the same session is instant — it's served from an in-session cache instead of calling the AI model again.

Reviewing the generated SQL

Before anything runs against your database, the app shows you:

- **Retrieved schema context** — an expandable section listing which tables the app decided were relevant, each with a similarity score.
- **Generated SQL** — the actual query, in an editable text box. Change it by hand if you want to adjust something.

Nothing runs automatically. The SQL is only a proposal until you click **Confirm and Run**. Every click of that button — including after you've hand-edited the SQL — is independently checked again before it's allowed to

execute.

Confirm and Run

Click **Confirm and Run** to execute the query. Results appear below as a table. Two checkboxes control how results are shown:

- **Show technical columns** — off by default. Turn on to see raw column names and internal ID/key columns that are normally hidden.
- **Show chart** — draws a chart from the result when the data shape supports one (a category or date alongside a number): a line chart for trends over time, a bar chart for comparisons across categories. It's disabled automatically when a result can't be meaningfully charted — a table-only result is normal and expected for many questions.

Large results are automatically capped at a maximum row count to keep the app responsive — if you need a smaller, more specific slice of data, narrow your question (add a filter, a date range, or a "top N").

Understanding the retry timeline

Sometimes the first SQL attempt doesn't work — a typo, a reference to the wrong column, and so on. When that happens, the app automatically tries again with the specific error in hand, up to a limited number of attempts, rather than giving up immediately.

Whenever more than one attempt happened, a **Retry timeline** section appears. Expand it to see exactly what happened on each attempt:

You'll see	What it means
✓ succeeded	This attempt worked; its results are what's shown.
✗ syntax error / missing reference	The SQL had a mistake or referenced the wrong table/column; the app corrected course and tried again.
● safety violation	The app blocked itself from running something other than a read-only lookup. This never retries — it's a hard stop, by design.
⌚ timeout	The query was taking too long and was stopped. This also doesn't retry automatically.

What if it fails?

If the app can't produce a working answer after its allotted attempts, you'll see a clear error message — never a blank screen or a crash — along with the last SQL it tried and a plain-language explanation of why it stopped. Common reasons:

- The question referenced data that genuinely isn't in the database.
- The question was too complex or ambiguous for the app to resolve confidently.
- The query was too expensive to run and timed out — try narrowing the question (smaller date range, an added filter, fewer things compared at once).

Under the Hood: Retrieval, Retries & Safety

This chapter goes one level deeper than Chapter 2 for readers who want to understand *why* the app behaves the way it does — still written for a general audience, not just developers. For the full code-level walkthrough, see `docs/ARCHITECTURE.md`.

Why the AI never sees the whole schema

A real production database can have hundreds of tables. Showing all of them to the AI model on every question would burn context and increase the chance of it inventing joins against irrelevant tables. Instead, each table's structure is pre-indexed (via ChromaDB), and only the small number most relevant to your specific question — plus any tables needed to structurally connect them — are shown to the model.

Filling the gaps: connecting the dots between tables

Pure text-similarity search has a blind spot: a table full of numbers (like a sales fact table) doesn't "read" as similar to a question like "which region had the highest sales," even though it's the one table that actually connects "region" to "sales." The app compensates for this automatically — it looks at how the initially retrieved tables are related in the real schema and pulls in any missing connector tables needed to bridge them, so the model isn't left inventing a shortcut that doesn't exist.

Fixing "the column name lied to me"

Some columns look, from their name and type alone, like they could hold almost anything — but actually hold short internal codes (e.g. `M / R / S / T`) rather than a business term like "Bikes." The app samples a handful of real distinct values from qualifying columns (short, low-cardinality text columns only — never a key column, and never a column with enough distinct values to plausibly be personal information) and shows those real examples to the model, so it filters on values that actually exist rather than a plausible-sounding guess.

Business context, kept current

A hand-maintained notes file describes what each table means and how it relates to others in plain language — including flagging columns that are commonly misread (like the coded example above). This file is re-read fresh on every single question rather than baked into the search index, so a correction to it takes effect on the very next question with no rebuild step.

The self-correction loop: not every failure is treated the same

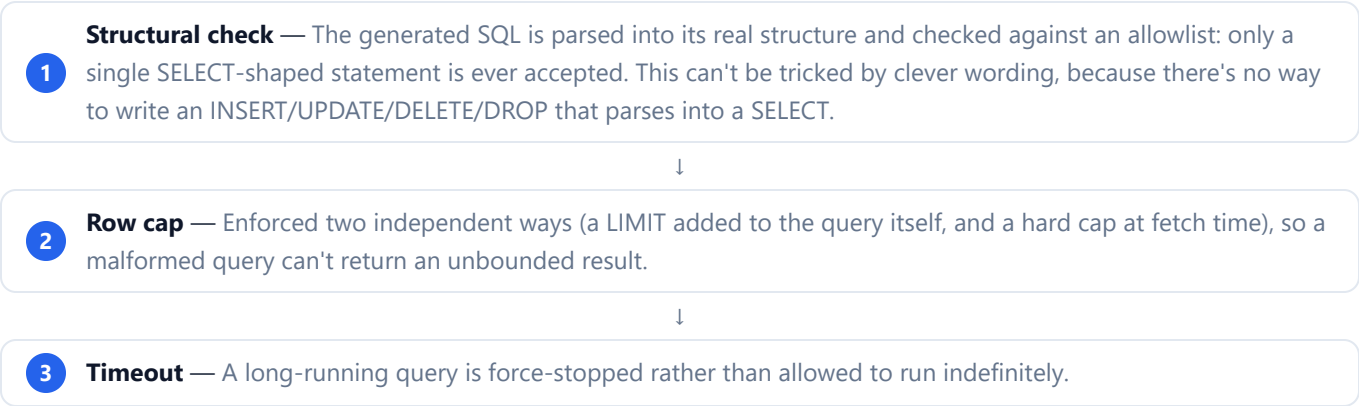
When something goes wrong, the app doesn't just blindly retry — it reacts differently depending on *why* the attempt failed:

What went wrong	Retries?	Why
Ordinary mistake in the SQL text	Yes	The model had the right information, it just wrote something slightly wrong — worth another attempt with the specific error shown.

Referenced a table/column that doesn't exist	Yes, differently	This can mean the <i>wrong tables were retrieved</i> in the first place, so the app goes back to the retrieval step (with the error text folded in as an extra search signal) rather than just regenerating against the same context.
Safety violation (attempted a non-read-only statement)	Never	A hard security stop, not a mistake worth coaching through — the app fails closed immediately, no matter how many retries remain.
Query timed out	Never	Retrying an expensive query with roughly the same shape wastes the whole retry budget on something a retry can't fix.

Every attempt — successful or not — is recorded so the Chapter 5 "Retry timeline" can show exactly what happened, rather than the process being invisible in a log file somewhere.

How "read-only" is actually enforced



This check runs *every time* SQL is about to execute — including after you hand-edit the SQL box yourself. A hand-edit is treated exactly as untrustingly as anything the AI produced.

Security & Data Privacy

What's actually guaranteed

- Generated SQL is restricted to a single read-only lookup via a structural (not keyword-based) allowlist, checked on every attempt, retry, and hand-edit.
- Query execution is row-capped and time-limited, enforced independently at two layers.
- The app never logs a connection string, password, or full result row — only the database type, name, table names, and row counts.
- The AI model runs entirely on your own machine. No schema, question, or result data is ever sent to a hosted API.

What is *not* guaranteed

The validator does not physically strip write privileges from the database connection itself — there's no generic way to do that at the connection layer across every supported database. The real guarantee has two layers: the SQL allowlist (described above), *and* the database login configured in `.env` being a genuinely read-only account. Both matter — the allowlist is one layer, not the only line of defense.

- This project has not been through an independent security review, penetration test, or adversarial prompt-injection testing.
- It is not designed for multi-tenant or production deployment — there's no authentication, no per-user authorization, and no rate limiting.

Bottom line: do not point this at a production database, or a database containing real sensitive/personal data, without an independent security review and your own risk assessment first. For local exploration, demos, or a sandboxed/synthetic dataset, a read-only database account plus the SQL allowlist is a reasonable posture. For anything more sensitive, treat this as a starting point, not a finished answer.

Setting up a read-only database account

Create a dedicated database login with `SELECT` -only privileges (no `INSERT` / `UPDATE` / `DELETE` / `DDL`) and point `DB_USER` / `DB_PASSWORD` in `.env` at it. This is the single most important thing you can do to harden this app beyond its built-in checks — see `SECURITY.md` for more detail.

Testing & Quality Checks

Automated unit tests

The mocked test suite runs without a real database or Ollama connection, covering the SQL validator, connection handling, schema introspection, schema retrieval, and the agent's node logic.

```
pytest  
# or: .\tasks.ps1 test
```

Linting & formatting

```
ruff check . && black --check . && mypy .  
# or: .\tasks.ps1 lint
```

Accuracy evaluation harness

Separate from the unit-test suite, an evaluation harness runs a growable set of real natural-language questions against a live database and a live Ollama model, and checks each answer against machine-checkable expectations:

- **Minimum row count** — did the query return a plausible amount of data?
- **Readable result** — is the first result column something a person can read, not a bare internal ID?
- **Correct tables used** — does the executed SQL actually reference every table the question logically requires?

That last check matters more than it sounds: a query can skip a required join step and still return a plausible, non-empty, readable result by coincidence — row count and readability alone won't catch that; explicitly checking which tables were used will.

```
python scripts\run_eval.py
```

This harness is what makes a join or filter regression show up as a failing check the next time it's run, instead of shipping silently and only being noticed when someone gets a subtly wrong answer.

Known Limitations

- **Accuracy depends heavily on the local model.** The default model handles straightforward aggregation/filter/join questions well but is noticeably weaker on window-function-heavy or deeply nested subquery questions. Swapping to a SQL-specialized model, or a larger one, generally helps at the cost of speed/memory.
- **Local inference is slower than a hosted API**, especially across several retry attempts on a hard question — this trades latency for running entirely offline with no per-call cost.
- **Schema retrieval can still miss a required table** on schemas with long, multi-hop hierarchies if more than one connecting hop is missing at once from the initial match. The automatic table-bridging (Chapter 6) catches the common case, not every possible gap.
- **Single-user, local-dev oriented.** This has not been hardened for concurrent multi-tenant use or production deployment — see Chapter 7 before pointing it at anything sensitive.

Troubleshooting

Symptom	Likely cause & fix
Red "Database Connection Required" screen on launch	The database couldn't be reached. Run <code>python scripts\test_db_connection.py</code> for a detailed diagnostic, and double-check <code>DB_TYPE</code> / <code>DB_HOST</code> / <code>DB_PORT</code> / <code>DB_NAME</code> in <code>.env</code> .
Connection fails with an authentication error	Check <code>DB_USER</code> and <code>DB_PASSWORD</code> — this is reported as a distinct category from "host unreachable" so you know which to check first.
Connection fails, host unreachable	Check <code>DB_HOST</code> / <code>DB_PORT</code> , VPN connectivity, and firewall rules.
"Driver missing" error, SQL Server (<code>DB_TYPE=mssql</code>)	The Microsoft ODBC Driver for SQL Server must be installed as a <i>system</i> package (not pip-installable) — install it, then confirm the name in <code>DB_ODBC_DRIVER</code> matches an installed driver.
App seems to hang on the first question	Ollama may still be loading the model into memory for the first call — this is normal and only happens once per model per Ollama restart.
"No tables discovered" / empty schema browser	Check <code>DB_SCHEMA</code> in <code>.env</code> — if set, only that schema is introspected. Try clearing it to use the connection's default schema, or click Refresh Schema after fixing it.
PowerShell won't run <code>Activate.ps1</code>	Check <code>Get-ExecutionPolicy</code> as yourself (not admin) and set it to <code>RemoteSigned</code> for the <code>CurrentUser</code> scope — avoid a machine-wide <code>Unrestricted</code> policy.
A pinned dependency fails to install	Likely a Python-version/wheel mismatch — see the Python version note in Chapter 4.

Frequently Asked Questions

Can this ever change or delete my data?

No. Every query is restricted to read-only lookups before it's allowed to run — see Chapters 6–7 for exactly how that's enforced.

Why does it sometimes take a few extra seconds?

That's usually the retry loop working (Chapter 5) — the app is making sure you get a correct answer rather than a fast wrong one. The very first question after starting Ollama can also be slower while the model loads.

The sidebar says the database is disconnected. What do I do?

Click **Test Connection** to confirm and see the specific reason. If it's still failing, see Chapter 10 or contact whoever manages the database configuration — this isn't something to fix from within the chat.

I asked the same question twice — why was the second one instant?

The app remembers answers to identical questions within your current session, so repeating one doesn't call the AI model again.

Can I trust the chart?

The chart is a best-effort visual of the table above it — always treat the table as the source of truth, and the chart as a convenience.

Which databases are supported?

PostgreSQL, MySQL, SQL Server, and Oracle, selected via `DB_TYPE` in `.env`. Switching databases is a configuration change, not a code change.

Can I change which AI model is used?

Yes — set `OLLAMA_MODEL` in `.env` to any model you've pulled into Ollama (e.g. a SQL-specialized model like `sqlcoder`). No code changes needed.

Does any of my data or my questions get sent anywhere?

No. The AI model runs locally via Ollama, the schema search index runs locally via ChromaDB, and the database connection is direct — nothing about your schema, your questions, or your results is sent to a third-party service.

Glossary

Term	Plain-language meaning
LLM	Large Language Model — the AI model that turns your question into SQL. Runs locally here via Ollama.
Ollama	The local software that runs the AI model on your own machine, no internet required.
LangGraph	The library used to build the app's explicit, step-by-step decision flow (Chapter 2), instead of a free-form AI agent.
ChromaDB	A local search index used to find which database tables are relevant to a given question.
Embedding	A numeric representation of text (a table's structure, or your question) that makes "how similar are these two things" a fast, computable operation — what powers the schema search.
SQLAlchemy	The library that connects to your actual database, regardless of which of the four supported engines it is.
sqlglot	The library that parses generated SQL into its real structure so it can be safety-checked, rather than scanned as plain text.
Allowlist	A security approach that only accepts things explicitly known to be safe (here: SELECT-only queries), rather than trying to block everything known to be dangerous.
Surrogate key	An internal ID column (e.g. <code>CustomerKey</code>) used to link tables together, with no business meaning of its own — hidden from the default results view.
Schema introspection	Reading a database's actual current tables, columns, and relationships directly from the database itself, rather than from a stored copy that could go stale.

Further Reading

This document is the comprehensive, user-facing reference. A few other files in the repository go deeper on specific areas:

File	What's in it
<code>README.md</code>	Quick-start overview and setup, aimed at a first-time visitor to the repository.
<code>docs/ARCHITECTURE.md</code>	The full code-level technical walkthrough: the LangGraph node design, retry/self-correction logic in detail, and the schema-retrieval pipeline internals.
<code>SECURITY.md</code>	This project's full security posture and how to report a vulnerability.
<code>CONTRIBUTING.md</code>	Developer setup, coding standards, and how to add a new evaluation question.

Text-to-SQL Dashboard — Documentation. Reflects the project as of its current state in this repository.